

DeskFlow 热桌预订 API — 架构与接口文档

目标

一个 Java REST API：管理办公室热桌，员工查看空闲桌子并按天预订。

- **数据库**：MySQL（应用 + demo），测试用 H2
- **核心规则**：一张桌子每天最多一个预订

一、架构

1.1 技术栈

- **Java 21 + Spring Boot 3.x**
- 依赖：spring-boot-starter-web、spring-boot-starter-data-jpa、spring-boot-starter-validation、mysql-connector-j(runtime)、h2(test)、（可选）springdoc-openapi-starter-webmvc-ui
- 构建：**Maven** (pom.xml)
- 持久化：**Spring Data JPA**

1.2 分层与包结构

标准三层 + DTO/实体分离（避免直接暴露 JPA 实体）：

```
src/main/java/com/deskflow/
├─ DeskflowApplication.java           # 启动类
├─ controller/                        # 只收发 DTO，定 HTTP 状态码
│   ├─ HealthController.java
│   ├─ DeskController.java
│   └─ BookingController.java
├─ service/                          # 业务规则、事务、实体↔DTO 转换
│   ├─ DeskService.java
│   └─ BookingService.java
├─ repository/                      # Spring Data JPA
│   ├─ DeskRepository.java           # JpaRepository<Desk, Long>
│   └─ BookingRepository.java        # JpaRepository<Booking, Long>
├─ entity/                          # JPA 实体
│   ├─ Desk.java
│   └─ Booking.java
├─ dto/
│   ├─ DeskResponse.java
│   ├─ BookingRequest.java           # 带 Bean Validation 注解
│   └─ BookingResponse.java
├─ exception/                       # 自定义异常
│   ├─ NotFoundException.java       # -> 404
│   ├─ BadRequestException.java     # -> 400
│   └─ ConflictException.java       # -> 409
└─ web/
    └─ ApiExceptionHandler.java     # @RestControllerAdvice, 统一错误 JSON
```

```
src/main/resources/
├─ application.yml          # 默认 profile = MySQL
├─ application-test.yml    # H2 (测试用)
└─ data.sql                # seed 数据 (或 CommandLineRunner)
```

1.3 中间件：需要哪些，不需要哪些

结论：不引入任何重量级中间件；只用 Spring 自带的 2 个轻量横切组件。

需求明确「不需要 auth / Docker / 微服务」，且是薄切片。因此：

❌ 不需要（引入只会拖慢、加环境复杂度、demo 时是负担）：

- 消息队列（Kafka / RabbitMQ）——没有异步/解耦需求
- 缓存（Redis）——数据量小，DB 直查足够
- API 网关 / 服务注册（Gateway / Nacos / Eureka）——单体应用，无微服务
- 独立鉴权中间件（Spring Security / JWT）——需求明确不要 auth

✅ 采用（Spring 自带，零额外依赖，成本低且是加分点）：

组件	技术	作用	是否必做
全局异常处理	<code>@RestControllerAdvice</code> + <code>@ExceptionHandler</code>	把 404/400/409 统一转成一致错误 JSON, controller 不写 try-catch	强烈建议做
参数校验	Bean Validation (<code>@Valid</code> + <code>@NotBlank</code> / <code>@NotNull</code>)	请求体自动校验, 非法参数由 advice 兜住返回 400	强烈建议做
请求日志	Servlet <code>Filter</code> 或 <code>HandlerInterceptor</code>	打印 method + path + 耗时, demo 控制台好看	可选 (有时间再加)
CORS	<code>WebMvcConfigurer</code> / <code>@CrossOrigin</code>	仅当做前端单页才需要	暂不做 (无前端)

`@RestControllerAdvice` 和 Bean Validation 是 Spring MVC 的横切机制，可视作应用内「中间件」。它们不是外部依赖，不需额外部署，是工程质量的关键体现点，演示时值得讲。

1.4 关键设计点

1. **核心并发规则**：`booking` 表对 `(desk_id, date)` 建**唯一约束**。Service 先查重返回 409，DB 唯一约束兜底防竞态。
2. **统一错误响应**：所有异常经 `ApiExceptionHandler` 转成 `{ "error": "...", "details": "..." }`。
3. **DTO 映射**：controller 只收发 DTO，service 做实体↔DTO 转换。
4. **Profile 隔离**：默认连 MySQL (`application.yml`)；集成测试 `@ActiveProfiles("test")` 用 H2 内存库、`ddl-auto=create-drop`。

1.5 数据模型

desk (实体 Desk)

字段	类型	约束
id	Long	PK, auto
code	String	UNIQUE 、非空，如 KRK-3F-12
floor	int	非空
has_monitor	boolean	默认 false
is_active	boolean	默认 true; false 不可预订

booking (实体 Booking)

字段	类型	约束
id	Long	PK, auto
desk_id	Long	FK → desk, 非空
employee_name	String	非空、非空白
date	LocalDate	非空
created_at	LocalDateTime	创建时自动填 (@CreationTimestamp 或 service 里赋值)

唯一约束: `UNIQUE(desk_id, date)` — 落实「一桌一天最多一预订」。

Seed 数据 (data.sql 或 CommandLineRunner)

- ≥8 张桌子，跨 2+ 楼层 (如 2F/3F)，部分 `has_monitor=true`，含 1 张 `is_active=false` 用于演示 400。
- 几条样例 booking，含今天/明天日期，方便 demo `GET /api/bookings?date=` 有数据。

二、接口文档 (逐参数)

通用约定

- Base path: `/api`
- 请求/响应体: `application/json; charset=utf-8`
- 日期格式: `YYYY-MM-DD` (`LocalDate`) ; 时间戳: ISO-8601 `YYYY-MM-DDThh:mm:ss`
- 统一错误响应体 (除 2xx 外均适用) :

```
{ "error": "<错误类别>", "details": "<可读说明>" }
```

#	方法 & 路径	说明	状态码
1	GET /api/health	健康检查	200
2	GET /api/desks	列桌子 (floor / hasMonitor 过滤)	200
3	POST /api/bookings	创建预订	201 / 404 / 400 / 409
4	GET /api/bookings?date=	查某天预订	200 / 400
5	DELETE /api/bookings/{id}	取消预订	204 / 404
C	GET /api/desks/available?date=	创意：查空闲桌 (待定)	200 / 400

接口 1 — 健康检查

GET /api/health

- 输入：无
- 输出 (200)：

字段	类型	说明
status	string	固定 "ok"

```
{ "status": "ok" }
```

- 预期结果：应用存活即 200。

接口 2 — 列出桌子

GET /api/desks

- 输入 (Query 参数, 均可选, 两个都实现)：

参数	类型	必填	说明	示例
floor	int	否	按楼层过滤	?floor=3
hasMonitor	boolean	否	按是否有显示器过滤	?hasMonitor=true

两个参数可组合：?floor=3&hasMonitor=true；都不传则返回全部。

- 输出 (200)：DeskResponse 数组, 每个元素：

字段	类型	说明
id	number	桌子 id

字段	类型	说明
code	string	唯一短码，如 KRK-3F-12
floor	number	楼层
hasMonitor	boolean	是否有显示器
active	boolean	是否可用

```
[
  { "id": 1, "code": "KRK-3F-12", "floor": 3, "hasMonitor": true, "active": true },
  { "id": 2, "code": "KRK-2F-05", "floor": 2, "hasMonitor": false, "active": true }
]
```

- 预期结果：始终 200；无匹配返回空数组 []（不是 404）。

接口 3 — 创建预订

POST /api/bookings

- 输入（请求体 `BookingRequest`）：

字段	类型	必填	校验规则
deskId	number	是	@NotNull
employeeName	string	是	@NotBlank（非空且非纯空白）
date	string(date)	是	@NotNull，格式 YYYY-MM-DD

```
{ "deskId": 1, "employeeName": "Anna Kowalska", "date": "2026-07-24" }
```

- 输出（201，`BookingResponse`）：

字段	类型	说明
id	number	预订 id
deskId	number	桌子 id
deskCode	string	桌子短码（demo 可读）
employeeName	string	员工名
date	string(date)	预订日期
createdAt	string(datetime)	创建时间

```
{
  "id": 10, "deskId": 1, "deskCode": "KRK-3F-12",
  "employeeName": "Anna Kowalska", "date": "2026-07-24",
  "createdAt": "2026-07-23T10:15:00"
}
```

• 预期结果 / 状态码矩阵:

场景	状态码	响应体示例
成功创建	201	上面的 BookingResponse
deskId 对应桌子不存在	404	{ "error": "Not Found", "details": "Desk 99 not found" }
桌子 is_active=false	400	{ "error": "Bad Request", "details": "Desk 3 is inactive" }
date 缺失/格式非法	400	{ "error": "Bad Request", "details": "date is required" }
employeeName 空白	400	{ "error": "Bad Request", "details": "employeeName must not be blank" }
该桌该日期已被订	409	{ "error": "Conflict", "details": "Desk 1 is already booked on 2026-07-24" }

• 校验顺序建议: 参数格式 (400) → 桌子存在 (404) → 桌子 active (400) → 是否冲突 (409) → 保存返回 201。

接口 4 — 查某天的预订

GET /api/bookings

• 输入 (Query 参数) :

参数	类型	必填	说明
date	string(date)	是	查询日期 YYYY-MM-DD

date 缺失或格式非法 → 400。

• 输出 (200) : BookingResponse 数组 (结构同接口 3 的输出) :

```
[
  { "id": 10, "deskId": 1, "deskCode": "KRK-3F-12",
    "employeeName": "Anna Kowalska", "date": "2026-07-24",
    "createdAt": "2026-07-23T10:15:00" }
]
```

• 预期结果: 该日期无预订返回空数组 []。

接口 5 — 取消预订

DELETE /api/bookings/{id}

- 输入（路径参数）：

参数	类型	必填	说明
id	number	是	预订 id

- 输出 / 预期结果：

场景	状态码	响应体
删除成功	204	无 body (或改 200 + { "message": "Booking 10 cancelled" })
预订 id 不存在	404	{ "error": "Not Found", "details": "Booking 99 not found" }

接口 C — 创意功能（待定，推荐首选）

GET /api/desks/available

团队自行拍板最终选型；本文档给出推荐实现，槽位已预留。

- 输入（Query 参数）：

参数	类型	必填	说明
date	string(date)	是	查询空闲的日期
floor	int	否	可叠加按楼层过滤

- 输出（200）：`DeskResponse` 数组（结构同接口 2），仅含该日期未被预订且 `active=true` 的桌子。
- 逻辑：查出该 date 已被预订的 `desk_id` 集合 → 返回 active 且不在集合中的桌子。复用 `DeskRepository` + `BookingRepository`。
- 预期结果：`date` 缺失 → 400；无空闲桌 → 空数组 `[]`。

其它备选创意（择一，接口形状供参考）：

- 我的预订：`GET /api/bookings?employeeName=X` → `BookingResponse` 数组
- 楼层统计：`GET /api/stats?date=X` → `[{ "floor": 3, "count": 5 }, ...]`
- 提前 N 天限制：在接口 3 加规则，`date` 超过今天+N 天 → 400

三、验证（端到端如何测）

1. 启动：`mvn spring-boot:run` → 连上 MySQL、seed 加载、`GET /api/health` 返回 `{"status":"ok"}`。

2. **手测矩阵** (curl / Postman, 覆盖所有状态码) :

- GET /api/desks?floor=3&hasMonitor=true → 200 过滤结果
- POST /api/bookings 合法 → 201
- 同 desk+date 再 POST → **409** (核心验收点)
- 不存在 deskId → 404; 停用桌 / 空 employeeName / 缺 date → 400
- GET /api/bookings?date=x → 看到刚建的预订
- DELETE /api/bookings/{id} → 204; 再删同 id → 404
- GET /api/desks/available?date=x → 返回空闲桌

3. **集成测试** (用 H2 profile, mvn test) : 至少 happy-path 建预订 + 409 冲突两条。

4. **对照需求 "Done means" 6 项逐条勾。**

四、Stretch (核心+创意跑通后按序)

统一错误 JSON (已纳入中间件, 做掉) → 集成测试 → DTO 映射 (已在架构内) → Swagger UI
(springdoc-openapi, demo 加分) → 前端单页 → Docker Compose。